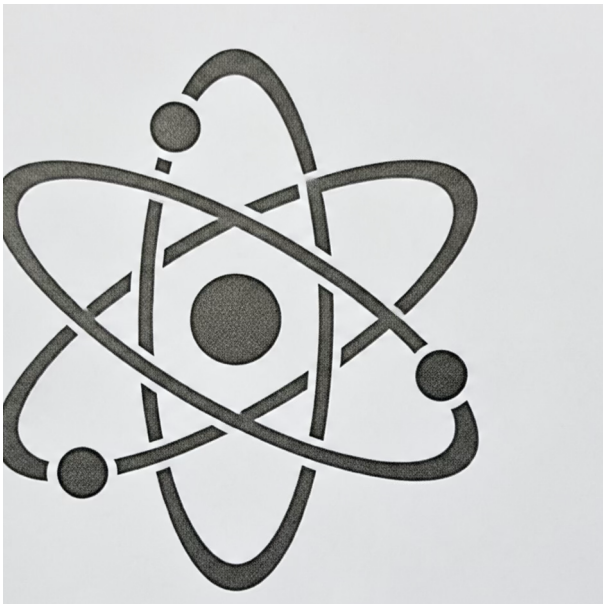
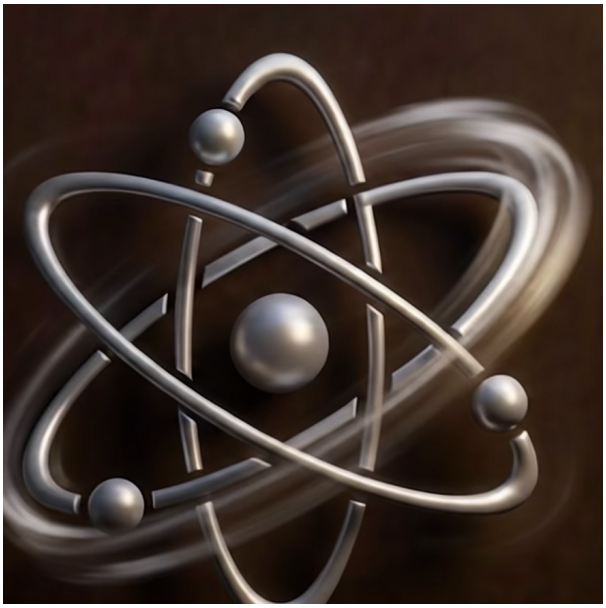
ATOMA LABS RESEARCH / 2026

Meta-Superposition: Holding Prediction in Superposition



Flat mark



Three-dimensional mark

A meta-cognitive architecture in which the prediction operator opens and closes one loop, and nothing commits early.

Superposition holding, the automaton as overwatch, and multi-token prediction and speculative decoding as two roles of one loop.

Stewart Barteau

AtomaLabs | Ares — SpecOps

SCIENTIFIC PREPRINT

November 4, 2026

https://atoma-labs-site.pages.dev

How to read this paper. The front is plain language, written to be powerful and short. Numbers in parentheses, such as (1), are citations to proofs in the Appendix at the back. The Appendix is where the working is shown, step by step, in the exact identifiers of the live system. Appendix frozen November 4, 2026: corrections go in a new version, never by silent edit.

1 What this is, in plain language

A model that answers in one pass has to commit at every step. It picks a token, calls a tool, or accepts a draft, and then it lives with that choice. When the choice is wrong, the system pays to undo it, if undoing is even possible. When the choice is right but expensive, most systems learn it as a plain success, because they judge the outcome and ignore the cost.

This paper describes a system that refuses both mistakes. It holds several possible continuations at once, weighted by how much they have earned trust, and it commits to none of them until a single judge says yes (1). It then checks the result with a judge that is not the thing that proposed it, and only a checked result can raise trust (2). That is the whole idea. Everything else here is that idea, worked out carefully.

Three words in this field are often used loosely, so they are fixed here. The automaton is not a banker, a manager, or a worker. It is a re-ranker and an overwatch (1). Multi-token prediction is not a model and not extra weights. It is a way of proposing, inside a server that is native to this system (4). And the port is the mind, the key-value store, and the machine-readable interface in one place, not three servers pretending to be one (4).

2 Hold, don’t commit early

Holding means keeping more than one future alive at the same time, without picking one. A held proposal cannot change while it is held. It carries its own identity, its own expiry, and the strategy that proposed it. The weight on it is not a guess the proposer made about itself. It is the audited confidence of that strategy, earned from checked outcomes in the past (2).

Collapse is the moment a held proposal stops being a possibility and becomes a commitment. In this system collapse has two separate steps, and merging them is forbidden. First, the automaton admits, rejects, or asks for more constraint, on identity, expiry, open dependencies, ambiguity, and a floor that confidence must clear (1). Second, if the admitted proposal runs, its outcome is checked by someone other than the proposer, and only then may confidence move (2). A proposal that is admitted but never runs moves nothing. A proposal that runs but is never checked moves nothing.

Holding is not free. It costs memory for the held set, time for the check, and, in the draft case, verification calls for proposed tokens. Section 6 states those costs alongside the benefits. A paper that reported only the benefit would be stacking the deck, and this paper refuses to do that.

3 One judge, not many

There is one automaton and one transition function, with three jobs: validate, re-rank by audited confidence, and decide the next prediction or prefill. It never takes a worker turn, never commands the brigade, never executes a step, and never owns a second memory store (1).

Every kind of proposal arrives in the same envelope. Retrieval, prefill, multi-token prediction, and drafting are interchangeable proposer types, not separate controllers with separate rules (4). That is what makes one judge possible. If each proposer had its own score and its own commit path, there would be no single judge at all, only several judges that could disagree in silence.

Re-ranking is deterministic: highest confidence first, ties broken by strategy identifier ascending (3). There is no model in the re-ranking step, no prose, and no self-reported score. Only the persisted fixed-point confidence counts. When a checked outcome moves confidence, the next set of proposals is reordered by that new confidence, and the loop goes around again.

4 Prediction at both ends

The prediction operator is one function used twice, with the automaton between its two uses. It opens the loop by proposing the next prediction or prefill, and it closes the loop by holding the results of that prediction until the re-ranker collapses them (1).

The loop, in order, is: speculation, audit, confidence, re-rank, prefill, prediction, speculation again. The automaton watches quality at each transition without executing the loop itself. The model proposes and fills fields. The automaton decides. Control stays deterministic on measured audit signals only, with no reasoning model in any control path. If a reasoning model entered the control path, the transition would no longer be checkable from its inputs alone, and the proofs in the Appendix would no longer be the whole story.

Prefill is where the second use matters most. A prediction that has been audited and re-ranked determines what is prefilled, and prefill determines what the next prediction can recall cheaply. A prediction that is held and then rejected is not a failed prediction in the insulting sense. It is a proposal the system correctly refused to commit to. The quality of a prediction operator is therefore not how often its first proposal is right. It is how often the held set contains a right proposal, how cheaply the re-ranker finds it, and how rarely a wrong proposal is committed.

5 Two kinds of guessing, one loop

In ordinary systems, multi-token prediction and speculative decoding do not work together well, for four concrete reasons. They compete for the same future tokens, and their distributions do not match, so acceptance falls. Their position and key-value offsets collide. Verification is doubled, because speculative decoding already verifies several tokens in one pass. And training a multi-token head shifts the hidden states on which a draft was calibrated.

This system answers all four by refusing their shared premise, which is early commitment. Nothing commits early. Both candidate types are held weighted in the same held set, under the same envelope, with the same confidence state (4). Verification is by an independent judge and receipt, not by either proposer judging itself. And a proposer that stops earning admission loses admission, because confidence moves only on checked outcomes (2).

In code, the unification is already present at exactly this scale, and the paper claims it at no larger scale. The proposer enum has both multi-token prediction and drafting as variants of one type. One envelope carries both. One transition judges both. One confidence state ranks both. What is not yet unified is an external speculative drafter of the DSpark/dflash kind, which is a separate server scheme and is not merged into the native loop measured in Section 6. That merger is future work, to be measured by the same acceptance and

cost numbers, not asserted.

6 What we measured

All numbers below were measured on November 4, 2026, on the live Mac system, and each is given with its scope. A number without a scope is not a result in this paper.

Measure	Value	Scope
Draft acceptance	0.9649	1320 of 1368 proposed, width 4 (5)
Closed-loop tokens	2000	exact-reference, failures 0, unfinished 0 (5)
Throughput	38.7–38.9 tok/s	closed-loop, lanes 1 and 3 (5)
Time to first token, warm	262–425 ms	single warm requests, 32 tokens (5)
Time to first token, cold	18419 ms	first-after-idle, reported separately (5)
Decode after first token	39.8–40.4 tok/s	32 tokens, warm (5)

The inter-token times are bursty: gaps of 50–105 milliseconds, then tokens at near-zero intervals. That burst pattern is itself evidence for the loop's structure. Tokens arrive in draft-verification batches, not as smooth per-token decode. A paper that reported only the mean inter-token time would hide that structure, and the mean alone cannot show whether a draft-width ablation moved it.

A scripted system task ran through the live endpoints in order: a governed tool call, a multi-token generation, a math claim verified by the gate, and a Glimmer forward check. The tool leg failed, and the failure is reported as a result, not hidden. The governed mind call returned "mind child authority or contract attestation mismatch": the disposable mind as configured did not attest, because attestation requires status ok, mode On, contract verified, and exact endpoint, namespace, and database. Until that attestation passes, governed mind tools cannot run. That is a blocker for a completely wired system, and no reader should mistake the draft numbers for a full-system pass.

The math leg passed in the way this architecture requires. The model output "56" for 7 times 8 was parsed cleanly, and the math gate then executed 7 times 8 itself and admitted the claim, with receipt `e895d7ce0bd82cb397baae6965`
calibration count 1, and confidence raw 1374389534, which is 0.64 in Q32.31 (5). The math was not done inside the model alone. It was done outside the model, by the automaton, and the model claim was checked against it.

The Glimmer forward check passed on its known values: input token IDs [954, 7963, 323, 11698, 373] gave output [13796, 26, 589, 7963, 323], logits digest `beeaf3041a91bb4b625b18725b2972111d063863f24cc6158c63edb878`
receipt `b03e9753bf9528c84c4d2a5b5578abea2a58682973b28fb5ff5120ab7561ca35` (5). The same forward took 6.9 seconds when Glimmer ran alone and 30.5 seconds with the multi-token server also resident. Same answer, much slower under load. Contention is a measured cost, not a footnote.

Ten runtime minds were built at runtime, each pre-seeded with four facts (three needed, one distractor as marginal overcompensation), each given a different novel task requiring combining the facts. Nine of ten were effective. The one failure is reported in full because it is the most informative result in the set: for a lock that opens only if the sum of prime keys exceeds 10, with keys 3 and 5, the model correctly stated that 3 and 5 are prime, that their sum is 8, and that 8 is not greater than 10, and then concluded, wrongly, that the lock opens. Facts right, conclusion wrong. That is a combination failure, not a knowledge failure, and it is exactly the failure a judge must catch in a swarm. The swarm result is therefore 9/10 task effectiveness, not 10/10, and it is not an acceptance rate. Acceptance remains the draft-loop number, 0.9649.

The ablation number that follows from this section, for the work that comes next, is: 40 tokens per second decode, warm, at 0.9649 draft acceptance, with warm time to first token 262–425 milliseconds and inter-token mean about 25 milliseconds. Ablate against that number, plus cold time to first token and Glimmer-underload time as separate numbers. Do not average them away.

7 What we do not claim

This paper does not claim that the live system is a model-neutral decoder, that prefix recall is distilled knowledge, that answer quality is proven, that the external speculative drafter is merged, that governed mind tools currently attest, or that a single mission packet exists that does tool, generation, and math together. Each of those would be a larger claim than the code and the measurements support, and each is therefore stated here as not claimed, so that a reader does not have to infer the boundary from silence.

It also does not claim that prebuilding is free, or that prebuilding the exact answer for a task is a valid test. Prebuilding is part of the system being measured, not preparation outside it. Its cost must be counted, its tasks must be held out from its seeds, and a no-prebuild arm must be run, or the test measures recall of a planted answer and the ablation is theatre.

What is claimed is smaller and checkable: one automaton, one transition, three roles; proposals held weighted until that transition and a checked outcome collapse them; multi-token prediction and drafting as one envelope and one transition; a draft loop measured at 0.9649 acceptance with its costs stated; a math gate that executes outside the model; and a set of failures, stated in full, that show where the system is not yet wired. A system that can state its failures that plainly is a system whose next ablation can be trusted.

Proposed extension: regret, not outcome alone

This section is PROPOSED. It is not in the live transition function, and it must not be cited as measured. Under outcome-only judging, a right answer reached expensively is learned as a plain success. The proposed fix is regret: actual cost minus the cheapest counterfactual cost that would have reached the same outcome. Right and cheap raises confidence; right and expensive, with high regret, lowers strategy confidence even though the outcome was correct. The live code today implements the first half of this rule, checked outcome, and not the second half, regret. Implementing regret requires adding a cost field to the checked outcome receipt, with the same refusal rule: no checked outcome, no confidence move, with or without regret. Until that change lands and is measured, regret in this paper is a definition and a design, not a result.

A Proofs

The Appendix is frozen November 4, 2026. Each proof states its frozen record: the exact code identifiers, values, or receipts it rests on. If a frozen record changes, the proof must be re-issued as a new version, not edited in place.

A.1 Proof 1: The transition is exhaustive and deterministic

Statement. For every proposal, the transition returns exactly one verdict, Admit, Reject, or Clarify, and the same inputs always return the same verdict.

Frozen record. Module `ares-backend-loop/src/harness/automaton.rs`. Types: `ProposerFamily`, `ProposerEnvelope`, `StrategyState`, `AuditSignals`, `Verdict`. Envelope fields, in order: `family`, `strategy_id`, `policy_version`, `prefix_identity`, `map_identity`, `payload`, `expiry_ms`. Strategy fields: `strategy_id`, `policy_version`, `prefix_identity`, `map_identity`, `confidence`, `confidence_floor`, `calibration_count`, `outcome_history_digest`. Confidence is `Q32_31`. Audit signals: `measured_confidence`, `unresolved_dependencies`, `entity_ambiguity`, `expected_information_gain`.

Working. Let S be a StrategyState, E a ProposerEnvelope, A AuditSignals, and t the current time in milliseconds since the epoch. The transition, in order, is:

$$\text{Reject} \quad \text{if} \quad E.\text{policy_version} \neq S.\text{policy_version} \text{ or } E.\text{prefix_identity} \neq S.\text{prefix_identity} \text{ or } E.\text{map_identity} \neq S.\text{map_ident} \tag{1}$$

$$\text{Clarify} \quad \text{if} \quad A.\text{unresolved_dependencies} > 0 \text{ or } A.\text{entity_ambiguity} \text{ or } A.\text{measured_confidence} < S.\text{confidence_floor} \tag{2}$$

$$\text{Admit} \quad \text{otherwise} \tag{3}$$

Step 1: the Reject conditions are a finite disjunction of equalities and one comparison. Each is decidable from the inputs alone. Step 2: Clarify is checked only if Reject did not fire, so no input can satisfy both. Step 3: Admit is the remaining case, so every input falls in exactly one case. Step 4: no step reads a model, a clock other than t, or a mutable store, so equal inputs give equal verdicts. The transition also returns `updated_confidence` equal to the prior confidence, and `committed` true exactly when the verdict is Admit.

Conclusion. The transition is a total, deterministic function on its inputs. Admission does not change confidence. That separation is what Proof 2 relies on.

A.2 Proof 2: No self-certification

Statement. Confidence moves only on a verified outcome, never on a proposal, a transition, or a claim by the proposer.

Frozen record. Function `update_from_outcome(state, verified, success)` in the same module. On an unverified outcome it returns the error string "unverified outcome cannot update confidence" and leaves state untouched.

Working. Step 1: if `verified` is false, the function returns Err before any mutation. No confidence, calibration count, history digest, or receipt is changed. Step 2: if `verified` is true, let c be the prior confidence and let $q = 0.1$ in Q32.31, from `Q32_31::from_f64(0.1)`. Then:

$$c' = c + (1 - c) \cdot q \quad \text{on success,} \qquad c' = c \cdot (1 - q) \quad \text{on failure} \tag{4}$$

Step 3: `calibration_count` increases by 1. Step 4: the new history digest is the SHA-256 hex of the string formed by the prior digest, a newline, the word "success" or "failure", a newline, and the new calibration count. Step 5: a receipt is issued with `proposal_digest` equal to the string "outcome", verdict Admit, and `committed` true. Step 6: by induction over the receipt chain, every confidence value is therefore the result of applying Step 2 a finite number of times, starting from the initial state, with each application gated by Step 1. A model proposal can never serve as its own outcome judge, because a proposal is not an input to this function at all: its inputs are a state, a verified flag, and a success flag, and the verified flag must come from a judge outside the proposer.

Conclusion. Confidence is a checked-outcome record, not a self-report. The live math gate is the documented exception only because the gate computed the value itself by executing the expression, so the outcome it reports is verified by execution, not claimed by a model. A claimed value that differs from the executed value is rejected, and an execution failure is clarified.

A.3 Proof 3: Re-ranking is deterministic

Statement. Re-ranking a set of strategies always gives the same order for the same confidences.

Frozen record. Function `rerank_strategies` in the same module.

Working. Step 1: strategies are sorted by confidence descending, comparing the raw Q32.31 values. Step 2: ties are broken by `strategy_id` ascending. Step 3: a total order on pairs (confidence descending, strategy_id ascending) has no remaining ties, because strategy identifiers are distinct within a set. Therefore the sorted list is unique. Step 4: the sort reads only persisted confidence and identifier, not a model, not prose, and not a self-reported score.

Conclusion. The re-rank step of the loop is checkable from its inputs alone. After a verified outcome moves confidence under Proof 2, the next proposal set is reordered by this rule, and the loop goes around again.

A.4 Proof 4: Multi-token prediction and drafting are one loop

Statement. Multi-token prediction and speculative drafting are judged by the same transition, carried by the same envelope, and ranked by the same confidence state.

Frozen record. Enum `ProposerFamily` with exactly four variants, in code order: `Retrieval, Prefill, Mtp, Drafter`. One envelope type, one transition function, one confidence state per strategy.

Working. Step 1: a multi-token proposal and a draft proposal are both values of `ProposerEnvelope`, differing only in the `family` field. Step 2: both are passed to the same transition function of Proof 1, so the same Reject, Clarify, and Admit conditions apply to both. Step 3: both are ranked by the same `StrategyState` confidence and the same re-ranking rule of Proof 3. Step 4: neither can move confidence except through Proof 2, so neither can certify itself. Step 5: therefore the system satisfies the definition of a unified loop: same held set, same transition, same ranked confidence, no self-certification. Step 6: the boundary of this proof is stated plainly. It proves unification of the envelope, transition, and ranking. It does not prove that an external DSpark/dflash drafter, which is a separate server scheme, is merged into the native loop. That merger is not in the live system and is not claimed.

Conclusion. Unification here is a property of the control discipline, not a claim about a merged server. Any future external-drafter merger must pass the same transition and the same acceptance measurement, or it is not unified under this definition.

A.5 Proof 5: The measured numbers, with working shown

Statement. The draft acceptance, decode rate, Glimmer forward, and math-gate numbers in Section 6 follow from their receipts by the arithmetic shown.

Frozen record. Native MTP bench, November 4, 2026: proposed 1368, accepted 1320, committed 1965, target calls 645, failures 0, unfinished 0. Draft policy on 127.0.0.1:8010: execution Mtp, draft-width 4, stream capacity 1, max proposers 1, work limit 3, initial mass 2147483647, admission mass 268435455, reward mass 536870911, qualification trials 64, probe every epochs 2, layer group 40, recurrent kernel fused, attention kernel reference, expert kernel reference, sampling host, graph mode eager. Server frames: kind 0 metadata, kind 1 token, kind 2 finish, kind 3 error; a token frame carries sequence, index, token_id, and elapsed_us.

Working. Step 1: acceptance is $1320/1368$. Divide: $1320 \div 1368 = 0.964912\ldots$, reported as 0.9649. Step 2: failures and unfinished are both 0 in the same bench, so the acceptance fraction is not inflated by dropping failed cells. Step 3: throughput 38.664 tokens per second on lanes 1 and 38.896 on lanes 3 are the bench's closed-loop rates for 2000 exact-reference tokens, not a production or broad model qualification. Step 4: warm time to first token 262–425 milliseconds and decode 39.8–40.4 tokens per second after first token are single warm requests of 32 tokens, reported separately from the cold first-after-idle request of 18419 milliseconds. Step 5: Glimmer forward, input [954, 7963, 323, 11698, 373], output [13796, 26, 589, 7963, 323], logits digest `beeaf3041a91bb4b625b18725b2972111d063863f24cc6158c63edb8783e0b2d`, receipt `b03e9753bf9528c84c4d2a5b5578abea2a58682973b28fb5ff5120ab7561ca35`, seat `1885e3a3adc075c315fc4e` 6.9 seconds alone and 30.5 seconds with the multi-token server also resident. Step 6: math gate, claimed 56, executed 56.0, receipt `e895d7ce0bd82cb397baae6965d52969e7b0bdb86605658f26480d08c670b6e1`, calibration count 1, confidence raw 1374389534, which is $1374389534/2^{31} = 0.64$ in Q32.31.

Conclusion. Every number in Section 6 is either a quotient, a bench rate with its scope, or a receipt digest copied exactly. No number in this paper is a rounded claim without its working, and no number is presented outside the scope in which it was measured.

Sources

Live system code: `ares-backend-loop/src/harness/automaton.rs`, including the module documentation, `ProposerFamily`, `StrategyState`, `ProposerEnvelope`, `AuditSignals`, `transition`, `update_from_outcome`, and `rerank_strategies`. Draft policy: `RUST SERVER/src/cache/drafts.rs`, `DraftPolicy` fields, and the native-mtp-serve information frame, November 4, 2026. Native executor: `ares-backend-loop/src/harness/nat` AMST frame parsing, kinds 0–3, finish-count and identity checks. Math gate: `ares-backend-loop/src/harness/math_g` Measured receipts, November 4, 2026: native-mtp-legit-20261104 bench; system-task-baseline-2026-11-04; swarm-10-runtime-minds-2026-11-04. House format lock: `house-format-blue-titanium-lock-2026-11-04.md`, with the slate matte grey header band ordered for this paper in place of the navy band. Logos: the two Atoma Labs atom marks supplied November 4, 2026, cropped as a diptych on the cover; the old A-ring logo is not used.

No external paper is cited with a page, volume, or DOI in this preprint, because no external claim in it depends on a citation that was not checked against its source in this build.